

Session 3 — Study Guide: Redux Toolkit in B4F Hub

This guide explains in depth what changed in B4F Hub in Session 3 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Before this session began, the Session 2 continuity homework (`RecentlyViewedContext`) was reviewed and its official solution taught live, so everyone started today's new content from the same place.

1. Where the session starts and ends

Start. Everything from Session 2, plus the recently-viewed feature. Three Contexts: `NotificationContext`, `SavedOpportunitiesContext`, `RecentlyViewedContext`.

End. Saved opportunities is managed by a Redux Toolkit slice inside one store. The other two Contexts are still there, unchanged. Your team also added a `clearAllSaved` action and three named selectors.

2. The question this session answers

You already have a tool for sharing state: Context. It works. So the question is not "is Context bad?" — it is: **for one particular concern, is there a tool that gives us something Context does not?**

Here is what Context gave us for saved opportunities:

TEXT

```
const { savedIds, toggleSaved } = useSavedOpportunities();
```

The component receives a **function** and calls it. Whatever that function does is defined somewhere else, and any component holding it can call it in any way it likes.

Redux changes the shape of that sentence:

TEXT

```
dispatch(toggleSaved(opportunity.id));
```

The component now **announces a fact** — "toggleSaved happened, for opportunity 101" — and one reducer, in one file, decides what that fact means for the state. The component does not know *how* saving works any more, and it cannot.

That is the whole trade. You write a little more code, and in exchange you get: one place where the update rules live, a named vocabulary of everything that can happen to this state, and a tool (Redux DevTools) that can show you every action that fired, in order.

3. The six words

- **store** — one object holding application state.
- **state** — the data inside it.

- **action** — a plain object describing something that happened: `{ type, payload }`.
- **reducer** — a pure function taking the current state and an action, returning the next state.
- **dispatch** — the one way to send an action to the store.
- **selector** — a function that reads a piece of state back out.

The flow runs one way only:

TEXT

```
UI → dispatch(action) → reducer → new state → selector → UI re-renders
```

Nothing in that loop is allowed to skip a step. A component never writes to the store directly.

4. Why saved opportunities, and not the other two

We migrated exactly one of the three Contexts in class, deliberately:

- **Saved opportunities** is read in three unrelated places (Navbar, the panel, each card) and changes through a small set of well-named events. That is application state.
- **Notifications** are a transient UI queue.
- **Recently viewed** is a small display list.

Both of those stayed in Context through the end of class today. **This is the answer to "when do I use which"**: it depends on the state and its updates — not on how big the app is, and not on whether a Redux migration is available.

A note on your homework. You will migrate `RecentlyViewedContext` to a second Redux slice as this session's continuity homework. That is **not** because Context was the wrong choice for it — it stayed in Context all through class for the reason above, and nothing about that reasoning changed. It is a deliberately chosen second practice feature, so that *you* repeat the Context-to-Redux pattern yourself, on a different concern, and can compare the same feature implemented both ways. In a real application, keeping a small concern like this in Context could still be a perfectly valid, permanent architectural choice — migrating it here is about your practice, not about a verdict on the design. `NotificationContext` gets no such homework: it stays Context, permanently, because a transient UI queue genuinely has no need for Redux's structure.

5. createSlice

TS

```
const savedOpportunitiesSlice = createSlice({
  name: "savedOpportunities",
  initialState,
  reducers: {
    toggleSaved(state, action: PayloadAction<number>) {
      const id = action.payload;

      if (state.savedIds.includes(id)) {
        state.savedIds = state.savedIds.filter((savedId) => savedId !== id);
      } else {
        state.savedIds.push(id);
      }
    },
  },
});
```

Three things are worth stopping on.

`state.savedIds.push(id)` is not a mutation bug. Redux Toolkit runs your reducers through a library called Immer. It hands you a draft, records what you do to it, and produces a brand-new state object. You get readable code and immutability at the same time. Outside a slice reducer, the old rules still apply — do not read this as permission to mutate state anywhere else.

You never wrote an action creator. `createSlice` generated `toggleSaved` from the reducer's name. Calling `toggleSaved(101)` returns `{ type: "savedOpportunities/toggleSaved", payload: 101 }` and changes nothing. Something must still dispatch it.

`PayloadAction<number>` is how TypeScript knows the action carries an opportunity id.

6. The store, and one thing about types

TS

```
const store = configureStore({
  reducer: {
    savedOpportunities: savedOpportunitiesReducer,
  },
});

type RootState = ReturnType<typeof store.getState>;
type AppDispatch = typeof store.dispatch;
```

The key `savedOpportunities` is why every selector starts `state.savedOpportunities`.

`RootState` is **derived**, not hand-written — it is literally "whatever `getState()` returns". When you add a second slice for the homework, `RootState` picks it up with nobody editing a types file. That is the type every `useSelector` call in this session actually uses.

`AppDispatch` is exported next to it because the two are conventionally written as a pair, but nothing in this session's code reads it — every `dispatch` call here is plain `useDispatch()`, which needs no type argument for a synchronous action like `toggleSaved`. `AppDispatch` becomes useful once you write a typed custom hook or dispatch asynchronous work, neither of which this session does. It is not a mistake that it sits unused today; it is a standard type prepared for later.

There is **one store** for the whole app. Features are slices inside it, never separate stores.

7. Reading and writing from a component

TEXT

```
const savedIds = useSelector(
  (state: RootState) => state.savedOpportunities.savedIds,
);
const dispatch = useDispatch();
```

`useSelector` subscribes the component to one piece of state and re-renders it when that piece changes. Select the **narrowest** thing you actually need — selecting the whole store means re-rendering on every unrelated change.

8. Serializable state, and why the Set disappeared

The Context held a `Set<number>`. The slice holds a `number[]`.

Redux state should stay **serializable** — plain data that could be written to JSON and read back. A `Set` is not, and Redux Toolkit warns about it in development. That single decision is why `savedIds.has(id)` became `savedIds.includes(id)` and `savedIds.size` became `savedIds.length` everywhere.

9. Selectors (the teamwork)

Before your teamwork, three components each contained this string:

TEXT

```
state.savedOpportunities.savedIds;
```

That is knowledge about the store's shape, copied into three files. Named selectors move it into one:

TS

```
export function selectSavedCount(state: RootState) {
  return state.savedOpportunities.savedIds.length;
}

export function selectIsSaved(state: RootState, id: number) {
  return state.savedOpportunities.savedIds.includes(id);
}
```

Rename the slice key now, and exactly one file changes.

Note the difference in how they are used. `selectSavedCount` takes only the state, so it can be passed straight in:

TSX

```
const savedCount = useSelector(selectSavedCount);
```

`selectIsSaved` needs an extra argument, and `useSelector` only ever passes the state, so it gets wrapped:

TSX

```
const isSaved = useSelector((state: RootState) =>
  selectIsSaved(state, opportunity.id),
);
```

10. Choosing between four tools

In the order you reach for them:

1. **Local state** — one component owns it. `savedOnly` in the Opportunities panel is still `useState`, and that is correct.
2. **Props** — explicit parent → child, short distance, same feature.
3. **Context** — many components in a subtree, or a sibling that props cannot reach.
4. **Redux** — an application-state concern where you also want one place for the update rules and a record of what happened.

What is **not** on that list: the size of the app. "Context for small apps, Redux for big apps" is the shortcut this session exists to replace — this app uses both, at the same time, for different concerns.

11. Redux does not remember anything

Save three opportunities, then reload the page. They are gone.

This surprises people, because "store" sounds permanent. It is not — it is an object in memory, exactly like the Context state was. Neither Context nor Redux persists anything by itself.

If data must survive a refresh, that is a separate, deliberate decision: `localStorage`, or a real backend. We will do the backend version later in this block. Do not reach for a persistence library now.

12. What you should be able to explain now

- The six words, and the one-way flow between them.
- Why `dispatch(toggleSaved(id))` is a different kind of sentence from `toggleSaved(id)`.
- Why `push` inside a `createSlice` reducer is safe, and where that stops being true.
- Why the saved `Set` became an array.
- Why `NotificationContext` is still in the app on purpose.
- What a selector buys you, and why `selectIsSaved` needs wrapping but `selectSavedCount` does not.
- Why reloading the page empties the saved list, and the two real ways to change that.

13. Not yet

These exist and are **not** part of this session. If you meet them online, park them:

- `createAsyncThunk` and `extraReducers` — Redux with asynchronous work.
- RTK Query — data fetching built on Redux.
- `createSelector` — memoized selectors.
- Custom middleware, normalized entities, persistence libraries.

Glossary

Word	Meaning
Store	The single object holding application state.
Slice	One named piece of the store: its state, its reducers, its selectors.
Action	A plain object describing something that happened.
Action creator	A function that builds an action object; <code>createSlice</code> generates them.
Reducer	A pure function from (state, action) to the next state.
<code>dispatch</code>	Sends an action to the store.
Selector	A function that reads a piece of state out of the store.
Immer	The library that makes draft mutations inside reducers safe.
Serializable	Data that could be written to JSON and read back — plain objects, arrays.

Review questions

1. What does `toggleSaved(101)` return, and what happens if you never dispatch it?
2. Why is `state.savedIds.push(id)` acceptable inside a slice reducer but not in a component?
3. The Navbar selects `selectSavedCount` instead of the array. Give two reasons.
4. Why did `savedIds.has(id)` become `savedIds.includes(id)`?
5. Name one thing in this app that should stay `useState`, and say why.

6. Why is `NotificationContext` still Context after this session?
7. What happens to the saved list on refresh, and what would you change if the product needed it to survive?
8. `selectSavedCount` can be passed directly to `useSelector`, but `selectIsSaved` cannot. Why?